ℹ This manual is intended for administrators only. End users should not run any script from the admin/ folder.

Administrator Manual

R and Python Validated Environment Setup

JR Scripts — Version 1.1

1. Overview

This manual describes how to set up and maintain the validated R and Python environments used by the JR Scripts project.

The key principle is a strict separation of responsibilities:

- The administrator sets up the local package repository and installs the environments.
- End users only run scripts — they never install packages or modify the environment.
- All packages are installed from a controlled local repository (e.g. Dropbox folder), never directly from the internet.

1.1 Architecture Overview

The following components together constitute the validated environment:

Component	Location	Purpose
Local R repo	R_repo/my-cran-repo	Controlled store of pinned R packages (miniCRAN). Installed and maintained by the admin. Shared via Dropbox.
Local Python repo	Python_repo/my-repo	Controlled store of pinned Python packages. Shared via Dropbox.
R library	~/.renv/[PROJECT_ID]/library/	Installed R packages for all user scripts, isolated per project.
Python venv	~/.venvs/[PROJECT_ID]/	Installed Python packages for all user scripts, isolated per project.
renv.lock	admin/	Exact version manifest for R packages, written by admin_install_R.
R_requirements.txt	admin/	Human-readable pinned R package list (user-installed packages).
R_base_requirements.txt	admin/	Pinned R packages available in the R installer (base packages).
python_requirements.txt	admin/	Human-readable pinned Python package list (user-installed packages).
python_base_requirements.txt	admin/	Python packages that are part of the standard Python installation.
r_version.txt	admin/	Specifies required R version (e.g. 4.5).
python_version.txt	admin/	Specifies required Python version (e.g. 3.11.9).
project_id.txt	admin/	Defines the project name used for R

		library path, Python venv path, and run log location (~/.jrscript/[PROJECT_ID]/). Must be set before running admin_install_R or admin_install_Python.
setup_jr_path.zsh	project root	Run once by each new user to add bin/ and wrapper/ to their shell PATH.

1.2 Folder Structure

The JR environment has the following folder structure:

Folder / File	Contents
bin/	JR infrastructure scripts: jrrun, jr_versions, jr_uninstall.
wrapper/	User entry point scripts: jr_animate, jr_static, and one wrapper per community script.
help/	Per-script help text files (same base name as R/Python script, .txt extension).
R/	User R scripts.
Python/	User Python scripts.
admin/	Admin-only scripts and configuration files (chmod 750 — users cannot read).
admin/R/	Admin R scripts (auto-generated validation scripts live here).
admin/Python/	Admin Python scripts (auto-generated validation scripts live here).
R_repo/	Local R package repository (controlled, Dropbox-synced, not in Git).
Python_repo/	Local Python package repository (controlled, Dropbox-synced, not in Git).
docs/	Admin manual, validation documents, regulatory justification.

1.3 Admin Scripts

Script	Purpose
admin/admin_install_R	Sets up the R environment from the local repo.
admin/admin_install_Python	Sets up the Python environment from the local repo.
admin/admin_create_hash	Generates SHA256 integrity file covering all project scripts.
admin/admin_validate	Generates and runs IQ validation scripts, produces timestamped evidence file.
admin/admin_uninstall	Removes the entire JR environment from this machine.
bin/jr_uninstall	Removes the current user's local environment components (R library, venv, run log, PATH entry).
bin/jr_versions	Displays current R version, Python version, and all installed package versions.
setup_jr_path.zsh	Run once by each new user — adds bin/ and wrapper/ to PATH via ~/.zprofile.

2. Prerequisites

2.1 Required Software

The JR environment is intended to be installed in a shared (e.g. Dropbox) folder to which all team members have restricted access. Before running any admin script, ensure the following software is installed on the admin machine:

Software	How to install
R (correct version)	Download from https://cran.r-project.org/bin/macosx/ — install the version matching r_version.txt.
Python (correct version)	Download from https://www.python.org/downloads/macos/ — install the version matching python_version.txt.

2.2 Required Files

Ensure the following files are present in the admin/ folder with correct content before running any admin script:

- R_requirements.txt and R_base_requirements.txt — list of R packages with pinned versions.
- python_requirements.txt and python_base_requirements.txt — list of Python packages with pinned versions.
- r_version.txt — contains the required R version, e.g. 4.5.
- python_version.txt — contains the required Python version, e.g. 3.11.9.
- project_id.txt — contains the project name (e.g. MyProject). This value determines the R library path (~/.renv/[PROJECT_ID]/), the Python venv path (~/.venvs/[PROJECT_ID]/), and the run log location (~/.jrscript/[PROJECT_ID]/). It must be set correctly before running admin_install_R or admin_install_Python. See Section 3.6 for multi-architecture teams.

2.3 Requirements File Format

Both R_requirements.txt and python_requirements.txt use the same format: one package per line, version pinned with ==, comments with #.

Example R_requirements.txt:

```
# R package requirements
tolerance==3.0.0
e1071==1.7-17
ggplot2==4.0.2
```

Example python_requirements.txt:

```
# Python package requirements
numpy==1.26.4
pandas==2.1.4
scipy==1.11.4
```

⚠ Use the exact version string as CRAN or PyPI reports it, including hyphens (e.g. 1.7-17). Do not normalise hyphens to dots.

R_base_requirements.txt and python_base_requirements.txt list packages used in user scripts that are part of the base R or standard Python installation. These do not need version pins and are not installed by admin_install_R or admin_install_Python.

The requirements files shipped with the JR environment already contain the packages required by the core infrastructure scripts. Only add packages required by your own user scripts.

3. Setting Up the R Environment

3.1 What the Admin Script Does

The script admin_install_R performs the following steps:

- Reads project_id.txt to determine the project name used for library paths and logs.
- Detects the macOS architecture automatically using uname -m and sets R_MACOS_PLATFORM to big-sur-arm64 (Apple Silicon / M-series) or big-sur (Intel). No manual action is needed.
- Reads R_requirements.txt to get the list of pinned packages.
- If called with --rebuild: downloads all packages and dependencies from CRAN into the local R repo.
- Verifies the integrity of the local R repo using checksums.
- Writes renv.lock from the pinned versions in R_requirements.txt.
- Installs all packages into the isolated R library at ~/.renv/[PROJECT_ID]/library/ using install.packages(lib=lib_path) — never the system library.
- Verifies all installed versions match the requirements.

ℹ The R library is isolated per project in ~/.renv/[PROJECT_ID]/library/. All users on the same project share the same pinned versions.

3.2 First-Time Setup (Building the Repo)

Perform this step once when setting up for the first time, or when updating package versions. Requires an internet connection.

- Ensure Dropbox is running and synced on your Mac.
- Open Terminal (Applications > Utilities > Terminal).
- Navigate to the project root folder: cd .../jrscripts
- Run with --rebuild: ./admin/admin_install_R --rebuild

When finished you should see:

```
✅ renv environment successfully created
📦 R version  : R version 4.5.0
📂 Repo       : .../R_repo/my-cran-repo
📋 Packages   : tolerance, e1071, ggplot2
```

ℹ The first-time download may take a few minutes depending on the number of packages and internet speed.

⚠ Known limitation: --rebuild fetches the current CRAN versions at the time of the run, not the versions pinned in R_requirements.txt. The local repository is the version control artefact. Rebuilding with different versions requires re-validation.

3.3 Regular Setup (Installing from Existing Repo)

Use this when setting up on a new machine where the Dropbox repo already exists, or after wiping the R library.

```
./admin/admin_install_R
```

⚠ Always ensure Dropbox has fully synced before running admin_install_R. A partial sync will cause the repo integrity check to fail.

3.4 Adding a Single Package

To add one new package without rebuilding the entire repo:

```
./admin/admin_install_R --add packagename==version
```

This downloads the package and its dependencies from CRAN into the existing local repo, updates R_requirements.txt, regenerates checksums and renv.lock, then installs.

⚠ Known limitation: --add fetches the current CRAN version of the package's dependencies. If a dependency is already pinned in R_requirements.txt at a different version, the script will report a conflict and abort. Resolve the conflict manually before retrying.

3.5 Updating Package Versions

- Edit R_requirements.txt and change the version number.
- Update r_version.txt if the R version also changes.
- Run admin_install_R --rebuild to download the new version and rebuild the repo.
- Re-generate the project integrity file (see Section 6).
- Communicate the change to all team members — the first script run will auto-rebuild their environments.

3.6 Multi-Architecture Teams (Apple Silicon and Intel)

Background

R binary packages are architecture-specific. A package binary compiled for Apple Silicon (arm64) will not run on an Intel Mac (x86_64) and vice versa. The local miniCRAN repo stores binaries in architecture-specific subdirectories:

```
R_repo/my-cran-repo/bin/macosx/big-sur-arm64/contrib/4.5/  ← Apple Silicon
R_repo/my-cran-repo/bin/macosx/big-sur/contrib/4.5/        ← Intel
```

admin_install_R automatically detects the architecture of the machine it runs on (via uname -m) and builds or installs the correct binary tree. On a homogeneous team (all Apple Silicon or all Intel), no special action is required. A mixed team requires additional setup by the administrator.

The Problem with Mixed Teams

If the admin runs --rebuild on an Apple Silicon machine, only the arm64 binary tree is populated. Intel users pointing at the same LOCAL_REPO will find an empty bin/macosx/big-sur/ directory and the install will fail. A single shared repo can contain both binary trees, but this requires the admin to run --rebuild on both architectures — which in practice means two separate machines.

⚠ Do not assume that source packages (bin/src/) provide a fallback. The environment is configured with pkgType = 'binary' and install.packages.compile.from.source = 'never' to ensure reproducibility. Source compilation is not permitted in the validated environment.

Recommended Solution: Separate PROJECT_IDs per Architecture

The recommended approach for a mixed team is to maintain two separate project configurations — one per architecture — each with its own PROJECT_ID:

Architecture	project_id.txt	R library	R repo subfolder
Apple Silicon (arm64)	MyProject	~/.renv/MyProject/library/	bin/macosx/big-sur-arm64/contrib/4.5/
Intel (x86_64)	MyProject_Intel	~/.renv/MyProject_Intel/library/	bin/macosx/big-sur/contrib/4.5/

Admin Setup Procedure for Intel Users

Perform these steps on an Intel Mac:

- Edit admin/project_id.txt and change the value to the Intel project ID (e.g. MyProject_Intel).
- Ensure the Dropbox repo is synced.
- Run: ./admin/admin_install_R --rebuild
- The script will detect Intel architecture and populate R_repo/my-cran-repo/bin/macosx/big-sur/contrib/4.5/ in the shared Dropbox repo.
- Revert admin/project_id.txt to the original Apple Silicon value (e.g. MyProject) before committing, or maintain two separate project folders.
- Re-run admin_create_hash on each architecture after any change.

> ℹ If two separate project folders are maintained on Dropbox (one per architecture), each folder has its own project_id.txt and its own setup_jr_path.zsh. This is the cleanest arrangement: each user runs setup_jr_path.zsh from their architecture-specific folder.

Distributing to Intel Users

When onboarding an Intel user:

- Provide them with the Intel project folder path (or a separate clone/copy of the project with project_id.txt set to the Intel PROJECT_ID).
- Ask them to run setup_jr_path.zsh from that folder — this adds the correct bin/ and wrapper/ paths to their shell.
- Their R library will be built at ~/.renv/MyProject_Intel/ and their venv at ~/.venvs/MyProject_Intel/ the first time they run a script.

> ⚠ An Intel user who accidentally runs setup_jr_path.zsh from the Apple Silicon project folder will have the wrong project_id.txt on their PATH and will get architecture mismatch errors when jrrun tries to load the R library. If this happens, re-run setup_jr_path.zsh from the correct folder and open a new Terminal window.

4. Setting Up the Python Environment

4.1 What the Admin Script Does

The script admin_install_Python performs the following steps:

- Reads project_id.txt to determine the project name used for the venv path.
- Reads python_requirements.txt to get the list of pinned packages.

- If called with --rebuild: downloads all packages into the local Python repo.
- Verifies the integrity of the local repo using MD5 checksums.
- Creates a clean Python virtual environment (venv) at ~/.venvs/[PROJECT_ID]/.
- Installs all packages from the local repo using pip --no-index (no internet access).
- Verifies all installed versions match the requirements.

> i Python pins versions end-to-end: pip honours exact version pins (==) and installs no other version. Unlike R, Python wheels are generally cross-platform and do not require separate builds for Apple Silicon and Intel.

4.2 First-Time Setup (Building the Repo)

- Ensure Dropbox is running and synced.
- Open Terminal and navigate to the project root folder.
- Run with --rebuild: ./admin/admin_install_Python --rebuild

4.3 Regular Setup

```
./admin/admin_install_Python
```

4.4 Adding a Single Package

```
./admin/admin_install_Python --add packagename==version
```

pip will resolve and install all transitive dependencies at their pinned versions automatically.

4.5 Updating Package Versions

- Edit python_requirements.txt with the new version.
- Update python_version.txt if the Python version also changes.
- Run admin_install_Python --rebuild.
- Re-generate the project integrity file.
- Communicate the change to all team members.

5. Environment Variables Reference

The following environment variables are set automatically by the wrapper scripts. They should not be set manually in Terminal or shell profiles.

Variable	Description
LOCAL_REPO	Full path to the local Dropbox R or Python package repository.
BUILD_REPO	Set to true to download packages from the internet. Default: false.
VENV_PATH	Full path to the Python virtual environment: ~/.venvs/[PROJECT_ID]/.
RENV_HOME	Full path to the R library root: ~/.renv/[PROJECT_ID]/.
RENV_PATHS_ROOT	Must equal RENV_HOME. Controls where the R library is read and written.
R_MACOS_PLATFORM	Set automatically by admin_install_R based on machine architecture. Values: big-sur-arm64 (Apple Silicon) or big-sur (Intel). Determines the binary package subdirectory used in the local R repo.

R_LIBS	Must be unset before running R scripts. Unset automatically by jrrun.
R_LIBS_USER	Must be unset before running R scripts. If set, it can redirect package loading to the wrong library.
R_LIBS_SITE	Must be unset before running R scripts.

⚠ R_LIBS_USER in particular can silently redirect package loading to a different library if set in ~/.Rprofile or ~/.zprofile. jrrun unsets all three R_LIBS* variables before every script execution.

6. Project Integrity Management

6.1 What the Integrity File Does

The file admin/project_integrity.sha256 contains SHA256 checksums of all project scripts. Every time a user runs a script via jrrun, the checksums are verified. If any file has been modified, jrrun refuses to run and displays an error.

6.2 Generating the Integrity File

Run this command from the project root folder after any legitimate change to project files:

```
./admin/admin_create_hash
```

ℹ Re-generate the integrity file whenever you update, add, or remove any script in R/, Python/, wrapper/, help/, or bin/. Also re-generate after changing R_requirements.txt or python_requirements.txt.

6.3 Repo Integrity

The local Dropbox repos have their own integrity mechanism separate from the project integrity file. When --rebuild is run, a checksums.txt is written inside the repo. On every subsequent install, the checksums are verified before any package is installed.

If the repo integrity check fails:

- Check Dropbox sync status — a partial sync can cause false failures.
- If files were accidentally modified, rebuild the repo with --rebuild.
- Never manually edit files inside the repo folder.

7. Troubleshooting

7.1 Common Errors and Solutions

Error message	Solution
❌ **LOCAL_REPO environment variable is not set**	Always call scripts via their zsh wrappers, not directly with Rscript or python3.
❌ **Local repo not found at ...**	Ensure Dropbox is installed, signed in, and fully synced. Check that R_repo/ exists at the expected path.
❌ **Repo integrity check FAILED**	Dropbox may not have finished syncing. Wait and retry. If the problem persists, rebuild with --rebuild.
❌ **R version mismatch**	Install the correct R version from https://cran.r-

	project.org/bin/macosx/. Check r_version.txt.
❌ **Version mismatch detected**	The installed package version does not match requirements. Run admin_install_R again.
❌ **PROJECT INTEGRITY CHECK FAILED**	A project file was modified. If the change was intentional, re-run admin_create_hash.
❌ **renv library not found at: ...**	The R library has not been built yet or was deleted. Run admin_install_R.
❌ **RENV_PATHS_ROOT is not set**	The R script was called directly, not via jrrun. Always use the wrapper.
❌ **R_MACOS_PLATFORM is not set**	admin_R_install.R was called directly instead of via admin_install_R. Always use the zsh wrapper.
No module named 'X' (Python)	Script was called directly outside jrrun, or the venv has not been built. Run admin_install_Python.

7.2 Checking What Is Installed

To verify which packages are installed and from where, run:

```
bin/jr_versions
```

For a full IQ validation with evidence file:

```
./admin/admin_validate
```

7.3 Starting from Scratch

If the environment is badly broken and you want to start completely fresh:

```
# Remove R library
rm -rf ~/.renv/[PROJECT_ID]
# Remove Python venv
rm -rf ~/.venvs/[PROJECT_ID]
# Rebuild both environments
./admin/admin_install_R
./admin/admin_install_Python
```

8. Adding a New Team Member

When a new team member joins, they do not run the admin scripts. The admin performs these steps:

- Ensure the team member has Dropbox installed and has been added to the shared Dropbox folder with view-only access to R_repo/ and Python_repo/.
- Ensure they have installed the correct R and Python versions (see Section 2.1).
- Determine the team member's Mac architecture (Apple Silicon or Intel). Provide them with the setup_jr_path.zsh from the correct project folder — Apple Silicon users use the standard project folder; Intel users use the Intel project folder (see Section 3.6).
- Ask them to run setup_jr_path.zsh to add the JR environment to their PATH. They should drag the file to a Terminal window and press Enter, then open a new Terminal window.
- Ask them to run jrrun jrhello.R "Hello" and jrrun jrhello.py "Hello" once. These will automatically detect that the R library and Python venv are missing and rebuild them from the Dropbox repo on first run.

ℹ The first run on a new machine will take longer than usual because the R library and Python venv are being built from scratch. Subsequent runs are fast.

9. Access Rights

9.1 Principle

The JR environment enforces a strict separation between the administrator and end users. To make this separation enforceable, the admin/ folder is set to mode 750 (owner read/write/execute, group read/execute, others no access). Users can run scripts via wrappers but cannot read or modify requirements files, admin scripts, or the integrity file.

9.2 macOS Filesystem Permissions

Location	Mode	Notes
Project root folder	755	Admin: read/write/execute. Users: read/execute only.
bin/ and wrapper/	755	Scripts executable by all users, writable only by admin.
R/ and Python/	755	Users can read scripts but not modify them.
help/	755	Users can read help files but not modify them.
admin/	750	Admin only. Users cannot read requirements files, admin scripts, or the integrity file.
~/.renv/[PROJECT_ID]/	755	Must remain writable by the user — the auto-rebuild writes the R library here.
~/.venvs/[PROJECT_ID]/	755	Must remain writable by the user — the auto-rebuild writes the venv here.

9.3 Setting Permissions After Installation

After completing the installation, run the following commands from the project root folder as the admin user:

```
# Lock down the project folder (run from project root, as admin)
chmod -R 755 .
chmod -R 750 admin/
# Remove write access for non-owners
chmod -R o-w .
```

> **i** Re-run these permission commands after every install or update.

9.4 The Auto-Rebuild Exception

The R library (~/.renv/[PROJECT_ID]/) and Python venv (~/.venvs/[PROJECT_ID]/) are stored in each user's home directory. Users therefore have write access to their own library and venv, which is required for the auto-rebuild to work.

This is by design. The auto-rebuild reads exclusively from the controlled Dropbox repo (verified by checksum) and writes only to the user's own home directory. It cannot modify the shared project folder.

9.5 Dropbox Repo Access

The R_repo/ and Python_repo/ folders should be shared as view-only in Dropbox so that team members can sync them but cannot modify them:

- In Dropbox on the web, navigate to the shared folder containing the JR project.
- Set sharing permissions for R_repo/ and Python_repo/ to Can view for all non-admin members.
- The admin account retains Can edit access.

⚠ The admin/ subfolder should never be shared with end users. It contains requirements files and the integrity file. Sharing it would allow users to circumvent integrity checks.

9.6 Verifying Permissions

```
# Check project root
ls -la
# Confirm admin/ is 750
ls -la admin/
# Check bin/ and wrapper/ scripts
ls -la bin/ wrapper/
```

Expected output for bin/ and wrapper/ scripts: -rwxr-xr-x (owner rwx, group r-x, others r-x). For admin/ contents: -rwxr-x--- (owner rwx, group r-x, others none).

10. Validated Environment Summary

The following table summarises all components that together constitute the validated environment for FDA purposes.

Component	Controlled by	How it is validated
R version	r_version.txt	Checked by jrrun and admin_validate before running any script.
Python version	python_version.txt	Checked by jrrun and admin_validate before running any script.
R package versions	R_requirements.txt + renv.lock	Verified by admin_install_R after installation; confirmed by admin_validate.
Python pkg versions	python_requirements.txt	Verified by admin_install_Python after installation.
Package sources	Local Dropbox repo	Repo integrity checked via checksums.txt on every install.
R library isolation	~/.renv/[PROJECT_ID]/library/	Explicit lib= path in install.packages(); RENV_PATHS_ROOT checked by every R script at startup.
Python venv isolation	~/.venvs/[PROJECT_ID]/	pip --no-index from local repo; venv activated by jrrun before every Python script.
Project file integrity	project_integrity.sha256	Verified by jrrun before every script execution.
Internet access	--no-index / local_repo_url only	R and Python cannot reach CRAN or PyPI during normal operation.

ℹ For FDA submission purposes, renv.lock, R_requirements.txt, python_requirements.txt, and the admin_validate evidence file constitute the primary validation artefacts.

11. Adding User Scripts to the JR Environment

The JR environment hosts validated R and Python scripts used in medical device development. Because scripts are used in a regulated context, every addition must be followed by an integrity hash update and communicated to the team.

Adding a new script involves up to three steps. Only Step 1 is mandatory.

11.1 Step 1 — Place the Script (Mandatory)

Place the R or Python script in the correct subfolder:

- R scripts → place in R/
- Python scripts → place in Python/

The script is then immediately callable via jrrun:

```
jrrun myscript.R arg1 arg2
jrrun myscript.py arg1
```

jrrun automatically routes to the correct subfolder based on the file extension. No wrapper or additional configuration is needed for basic use.

11.2 Modifying R Scripts

Every R script must load its packages exclusively from the validated R library. Add the following block immediately before any library() calls:

```
# --- Load from validated R library ---
renv_root <- Sys.getenv("RENV_PATHS_ROOT")
if (renv_root == "") {
  stop("❌ RENV_PATHS_ROOT is not set. Run this script from jrrun.")
}
lib_path <- file.path(renv_root, "library")
if (!dir.exists(lib_path)) {
  stop(paste("❌ renv library not found at:", lib_path))
}
.libPaths(c(lib_path, .libPaths()))
```

Wrap all library() calls to suppress irrelevant startup messages:

```
suppressWarnings(suppressPackageStartupMessages({
  library(ggplot2)
  library(tolerance)
}))
```

11.3 Step 2 — Add a Help File (Optional but Recommended)

Create a plain text file in the help/ subfolder with the same base name as the script and a .txt extension:

- R script R/myscript.R → help file help/myscript.txt
- Python script Python/myscript.py → help file help/myscript.txt

When a user runs jrrun myscript.R --help, jrrun will display the contents of help/myscript.txt. Without this file, jrrun prints a generic message.

Suggested help file format:

```
Usage: jrrun myscript.R <input_file> <output_file>

Description:
  Brief description of what the script does.

Arguments:
  input_file    Path to the input CSV file
  output_file   Path for the output PNG file

Example:
  jrrun myscript.R data/input.csv results/output.png
```

11.4 Step 3 — Add a Wrapper (Optional)

A wrapper in wrapper/ provides a named shortcut for end users. This is optional — users can always call jrrun directly. Use a wrapper when:

- The script has a fixed, well-known purpose with a friendly name (like jr_animate, jr_static).
- The script takes no arguments or always the same arguments.
- You want the script easily discoverable without knowing the filename.

A wrapper is always a two-liner:

```
#!/bin/zsh
exec "$(dirname "$0")/../bin/jrrun" myscript.R "$@"
```

Make it executable and place it in wrapper/:

```
chmod 755 wrapper/mywrapper
```

Users can then call mywrapper directly from any directory, as both bin/ and wrapper/ are on the PATH.

11.5 Checking and Updating Package Requirements

Before finalising the script addition, verify that all packages used by the script are listed in the appropriate requirements file:

Package type	Where to add it
R user package (e.g. ggplot2)	R_requirements.txt using format: packagename==version
R base package (e.g. stats, MASS)	R_base_requirements.txt — no version needed
Python user package (e.g. numpy)	python_requirements.txt using format: packagename==version
Python stdlib module (e.g. os, sys)	python_base_requirements.txt — no version needed

> ℹ stats and MASS are base R packages that ship with R. They do not need to be added to R_requirements.txt.

> ⚠ Never add base R packages or Python standard library modules to the user requirements files. This can cause version conflicts and will trigger unnecessary downloads and installs.

11.6 Finalising the Script Addition

After placing the script and optionally adding a help file and wrapper, always complete these steps:

- Re-run admin_create_hash to include the new files in the project integrity file.
- If requirements files were updated: re-run admin_install_R or admin_install_Python with --rebuild, and re-run admin_generate_validate to update the IQ validation scripts.
- Inform team members. If package versions changed, their environments will auto-rebuild on next run.

⚠ Always re-run admin_create_hash after adding or modifying any script. jrrun will refuse to execute any script not covered by the integrity file.

12. Run Log

12.1 User Run Log

Every time a user runs a script via jrrun, a log entry is appended to:

```
~/.jrscript/[PROJECT_ID]/run.log
```

Each entry contains: ISO timestamp, script name (jrrun), script filename and arguments, exit code. Example:

```
2026-03-12T14:32:01  jrrun  jrhello.R Hello     0
2026-03-12T14:35:44  jrrun  jrhello.py Hello    0
```

12.2 Admin Log

Admin actions (admin_install_R, admin_install_Python, admin_validate) are logged to:

```
~/.jrscript/[PROJECT_ID]/admin.log
```

Format is the same as the run log with the admin script name and exit code or PASSED/FAILED status.

ℹ For FDA audit purposes, the run log and admin log provide a complete record of all script executions and environment changes. Back these files up regularly.

13. Important Note on Validation Scope

The validation evidence supplied with this distribution is specific to the R version, Python version, and package versions listed in the requirements files. Re-validation is required when:

- The R or Python version is changed in r_version.txt or python_version.txt.
- Any of the package version numbers as shipped is updated in R_requirements.txt or python_requirements.txt.
- The environment is installed on a new machine as part of a regulated workflow.

Revalidation is performed by executing the Statistics Validation Plan (docs/statistics_validation_plan.pdf, JR-VP-002) and recording the results in a new Statistics Validation Report.